8-Puzzle Problem Using Python (A* Search)

Aim

To write a Python program to solve the 8-Puzzle Problem using the A* Search Algorithm with the Manhattan Distance heuristic to find the shortest path from the initial state to the goal state.

Algorithm

1.Start the program.
2.Define the initial state and the goal state of the 8-puzzle.
3.Calculate the heuristic value (Manhattan Distance) for the current state.
4.Generate all possible moves by moving the blank tile (0) up, down, left, or right.
5.Select the next state with the lowest cost using the A* search algorithm.
6.Repeat the process until the goal state is reached.
7.Display the sequence of moves from the initial state to the goal state.
Stop the program.

Python Code

```
import heapq
goal = [[1,2,3],[4,5,6],[7,8,0]]
moves = [(1,0),(-1,0),(0,1),(0,-1)]
def h(s):
    return sum(s[i][j] != goal[i][j] and
s[i][j] != 0 for i in range(3) for j in
range(3))
def solve(start):
    pq = [(h(start), start, [])]
    seen = set()

    while pq:
        _, s, p = heapq.heappop(pq)

        if s == goal:
            return p + [s]
        seen.add(str(s))
        x, y = [(i, j) for i in range(3)
for j in range(3) if s[i][j] == 0][0]
        for dx, dy in moves:
            nx, ny = x + dx, y + dy
```

```
            if 0 <= nx < 3 and 0 <= ny <
3:
                t = [r[:] for r in s]
                t[x][y], t[nx][ny] =
t[nx][ny], t[x][y]
                if str(t) not in seen:
                    heapq.heappush(pq,
(h(t) + len(p), t, p + [s]))
start = [[1,2,3],[4,0,6],[7,5,8]]
for step in solve(start):
    print(*step, sep="\n")
    print()
```

EXP 1(8 Puzzle Slover).py | EXP 2(8 Queen Problem).py | EXP 3(Water Jug Problem).py | EXP 4(Cript-Arithmetic Pro

EXP 1(8 Puzzle Slover).py > solve

```python
import heapq
goal=[[1,2,3],[4,5,6],[7,8,0]]
moves=[(1,0),(-1,0),(0,1),(0,-1)]
def h(s): return sum(s[i][j]!=goal[i][j] and s[i][j]!=0 for i in range(3) for j in range(3))
def solve(start):
    pq=[(h(start),start,[])]
    seen=set()
    while pq:
        _,s,p=heapq.heappop(pq)
        if s==goal: return p+[s]
        seen.add(str(s))
        x,y=[(i,j) for i in range(3) for j in range(3) if s[i][j]==0][0]
        for dx,dy in moves:
            nx,ny=x+dx,y+dy
            if 0<=nx<3 and 0<=ny<3:
                t=[r[:] for r in s]
                t[x][y],t[nx][ny]=t[nx][ny],t[x][y]
                if str(t) not in seen:
                    heapq.heappush(pq,(h(t)+len(p),t,p+[s]))
start=[[1,2,3],[4,0,6],[7,5,8]]
for step in solve(start):
    print(*step,sep="\n"); print()
```

PROBLEMS | OUTPUT | DEBUG CONSOLE | TERMINAL | PORTS

```
PS E:\OPEN CV\AI Lab> & C:\Users\user\AppData\Local\Programs\Python\Python310\python.exe "e:/OPEN CV/AI La
[1, 2, 3]
[4, 0, 6]
[7, 5, 8]

[1, 2, 3]
[4, 5, 6]
[7, 0, 8]

[1, 2, 3]
[4, 5, 6]
[7, 8, 0]
```